

ExcelR

Full Stack Development Program

PROJECT REPORT

StayEvent – Hotel & Event Booking Management System

A Full Stack MERN (MongoDB, Express.js, React.js, Node.js) Web Application

Submitted by

Siddharth Singh

2026

Acknowledgement

I would like to express my sincere gratitude to the training team and mentors at ExcelR for their guidance and support throughout the course of this project, "StayEvent – Hotel & Event Booking Management System".

This project was undertaken as part of the Full Stack Development curriculum, and helped consolidate practical, hands-on experience with the MERN stack (MongoDB, Express.js, React.js and Node.js) by applying it to a complete, real-world style application.

I would also like to thank my family and friends for their constant support and encouragement during the completion of this project.

Siddharth Singh

Abstract

StayEvent is a full-stack web application built on the MERN stack (MongoDB, Express.js, React.js and Node.js) that allows customers to search, compare and book hotel rooms and event venues across India through a single, unified platform. The system is designed around an open-browsing model: any visitor can explore hotels, event venues, prices, live availability and guest reviews without creating an account, and is asked to sign in only at the final step of making a booking. Authentication is kept intentionally simple, using a short user ID and password instead of an email/OTP-based flow, to minimise friction for end users.

The platform supports two categories of bookable inventory – hotel rooms (with per-room-type stock and date-range availability) and event venues (booked per calendar day for weddings, conferences, birthdays and similar occasions). A booking passes through a well-defined lifecycle of Pending, Confirmed and Cancelled states, and unpaid bookings automatically release the held inventory after a fixed time window to prevent inventory being locked indefinitely. The system integrates a simulated multi-method payment gateway supporting UPI, credit/debit cards, net banking, mobile wallets and a pay-at-property (cash-on-delivery) option, complete with success and failure handling, so that the checkout experience mirrors a real production payment flow without requiring external API keys.

Additional features include a ratings-and-reviews module restricted to guests with a confirmed booking, role-based access control distinguishing regular users from administrators, and a full administrative dashboard for managing hotels, venues, bookings, payments, reviews and user accounts. The backend exposes a RESTful API built with Express.js and Mongoose over MongoDB, secured using JSON Web Tokens (JWT) and bcrypt password hashing, while the frontend is built with React (Vite) and Tailwind CSS to deliver a responsive, Booking.com-inspired user interface. This report documents the requirements, design, implementation and testing of the StayEvent system.

Keywords: MERN Stack, Hotel Booking, Event Venue Booking, React.js, Node.js, Express.js, MongoDB, JWT Authentication, Online Payment Simulation, Full Stack Web Application

Table of Contents

(Right-click the field below in Microsoft Word and choose "Update Field" → "Update entire table" to generate page numbers after opening this document.)

Chapter 1: Introduction

1.1 Overview

Online travel and event booking has become one of the most widely used categories of e-commerce, with customers expecting to search, compare and reserve accommodation or venues in a few clicks. Most existing platforms, however, treat hotel stays and event venues as separate products, forcing customers to use different applications for a trip and for a family or corporate event. StayEvent addresses this by combining both hotel and event-venue booking into a single MERN-stack web application, styled after leading travel platforms, while keeping the sign-up and login process deliberately lightweight.

1.2 Problem Statement

Existing academic and small-scale booking projects are often limited to a single type of inventory (only hotels, or only halls), lack a realistic payment experience, and force users to register before they can even view prices or availability. There is a need for a system that (a) lets visitors browse freely to encourage exploration, (b) supports both hotel rooms and event venues with correct availability logic for each, (c) offers multiple, realistic payment options with success/failure handling, and (d) gives administrators full control over the catalogue, bookings, payments and reviews.

1.3 Objectives

- To design and develop a unified platform for booking hotel rooms and event venues.
- To allow guests to browse listings, check prices and view reviews without mandatory registration.
- To implement a simple, low-friction authentication system using a user ID and password.
- To prevent double bookings through server-side, date-aware availability checks for both hotels and venues.
- To simulate a realistic multi-method payment gateway (UPI, card, net banking, wallet, pay-at-property).
- To provide a ratings-and-reviews system restricted to guests with a confirmed booking.
- To build a complete administrative dashboard for managing the platform end-to-end.

1.4 Scope of the Project

The scope of this project covers the complete customer journey – searching, filtering, viewing details, checking availability, booking, paying and reviewing – as well as the complete administrative journey of managing hotels, venues, bookings, payments, reviews and users. The project is a functional academic prototype: payments are simulated rather than processed through a licensed payment aggregator, and features such as SMS/email notifications, multi-language support and third-party map integration are identified as future scope rather than implemented in the current version.

1.5 Organization of the Report

Chapter 2 reviews existing systems and positions the proposed system against them. Chapter 3 presents the system analysis, including functional and non-functional requirements and feasibility. Chapter 4 covers the system design, including architecture, database design and modules. Chapter 5 lists the technology stack. Chapter 6 describes the implementation with representative code. Chapter 7 documents the testing carried out. Chapter 8 presents the results. Chapter 9 concludes the report and outlines future scope, followed by references and appendices.

Chapter 2: Literature Survey / Existing System

2.1 Study of Existing Systems

A number of well-established platforms were studied to understand common patterns in hotel and event booking systems:

- Booking.com and MakeMyTrip – large-scale hotel search platforms with destination browsing, filters, rating badges and a "reserve now, pay later" style flow. The visual layout and search-bar design of StayEvent are inspired by this class of product.
- EventEye / Vibe / local banquet-hall aggregators – focused specifically on event venues, with capacity- and date-based booking rather than the check-in/check-out model used for hotels.
- Generic e-commerce checkout flows – studied for patterns such as an inventory "hold" during checkout, multiple payment options, and order/booking status tracking (Pending, Confirmed, Cancelled).

2.2 Limitations Observed

- Hotels and event venues are almost always offered as separate products on separate platforms.
- Many require account creation before a user can even view detailed prices or availability, increasing drop-off.
- Small-scale academic projects in this domain frequently implement only a single payment method, or skip payment failure handling entirely.
- Double-booking prevention is often naive (e.g., a global "booked" flag rather than per-room-type, date-range-aware stock).

2.3 Proposed System

StayEvent is proposed as a single platform that combines hotel and event-venue booking, allows open browsing without an account, and implements date-range/day-based availability checks correctly for each inventory type. It also simulates a complete, multi-method payment gateway with a time-boxed inventory hold, so that the checkout experience closely mirrors a production system while remaining safe to run and demonstrate without real financial transactions or third-party API keys.

Chapter 3: System Analysis

3.1 Functional Requirements

ID	Requirement
FR-1	Visitors can browse hotels and event venues, view details, prices and reviews without signing in.
FR-2	Visitors can register and log in using only a user ID and a password.
FR-3	Users can search and filter hotels/venues by city, price, rating, star rating, amenities/facilities and capacity.
FR-4	The system checks and displays live room/venue availability for the dates selected.
FR-5	Users can create a booking, which places a 15-minute hold on the selected inventory.
FR-6	Users can pay via UPI, card, net banking, wallet, or choose pay-at-property.
FR-7	The system prevents double booking of the same room type/date range or venue/date.
FR-8	Users can view, cancel (with a 24-hour cut-off) and print/download their bookings.
FR-9	Users with a confirmed booking can post, edit and delete a review for that hotel/venue.
FR-10	Administrators can add, edit and delete hotels and venues, and manage room types.
FR-11	Administrators can view and update the status of all bookings and payments.
FR-12	Administrators can moderate reviews and manage user roles/accounts.

3.2 Non-Functional Requirements

Category	Requirement
Usability	Responsive, mobile-friendly interface; browsing requires no account.
Security	Passwords hashed with bcrypt; sessions secured with JWT; role-based route protection.
Performance	API responses for catalogue/search return within 1–2 seconds under normal load.
Reliability	Centralised error handling; booking/payment operations are validated server-side.
Scalability	Stateless REST API and document-oriented database allow horizontal scaling.
Maintainability	Modular MVC-style backend (models/controllers/routes) and component-based frontend.
Portability	Runs on any platform with Node.js and MongoDB; deployable to standard cloud hosts.

3.3 Hardware Requirements

- Processor: Dual-core 2 GHz or higher (development machine)

- RAM: 4 GB minimum, 8 GB recommended
- Storage: 500 MB free space for source code and dependencies
- Internet connection for package installation and cloud database access

3.4 Software Requirements

- Operating System: Windows 10/11, macOS or Linux
- Node.js v18 or higher and npm
- MongoDB (local installation or MongoDB Atlas cloud cluster)
- Code editor: Visual Studio Code
- Web browser: Google Chrome / Microsoft Edge / Mozilla Firefox
- Version control: Git

3.5 Feasibility Study

3.5.1 Technical Feasibility

The MERN stack is a mature, well-documented, open-source technology stack. All libraries used (Express, Mongoose, React, JWT, bcrypt) are freely available via npm, and the development team has adequate familiarity with JavaScript across both the client and server, making the project technically feasible.

3.5.2 Economic Feasibility

The entire stack is open source and free to use. MongoDB Atlas, Render and Vercel/Netlify all provide free tiers sufficient for development and academic demonstration, so the project can be built and deployed at zero monetary cost.

3.5.3 Operational Feasibility

The system is designed to be simple to operate: end users need only a browser, and administrators are given a dedicated dashboard for day-to-day operations such as adding new properties or managing bookings, requiring no technical/database knowledge.

Chapter 4: System Design

4.1 System Architecture

StayEvent follows a three-tier client–server architecture:

- **Presentation Tier:** A React.js (Vite) single-page application responsible for rendering pages, client-side routing and calling the backend REST API via Axios.
- **Application Tier:** A Node.js/Express.js REST API responsible for authentication, business rules (availability, pricing, tax, cancellation windows), and orchestrating data access.
- **Data Tier:** A MongoDB database, accessed through Mongoose object models, storing users, hotels, venues, bookings, payments and reviews.

Communication between the presentation and application tiers uses JSON over HTTPS/REST, and the application tier authenticates requests using stateless JSON Web Tokens, so the API itself holds no server-side session state.

4.2 Actors and Use Cases

- **Guest (unauthenticated visitor):** browse hotels/venues, search, filter, view details, availability and reviews, register, log in.
- **Registered User:** all guest actions, plus create booking, make payment, view/cancel own bookings, write/edit/delete own reviews, edit profile.
- **Administrator:** all user actions, plus manage hotels/venues (CRUD), view and update all bookings/payments, moderate reviews, manage user accounts and roles, view dashboard statistics.

4.3 Module Description

4.3.1 Authentication Module

Handles registration and login using a user ID and password (bcrypt-hashed), issuing a signed JWT on success and exposing profile read/update endpoints.

4.3.2 Catalogue Module (Hotels & Venues)

Provides public search, filter, city-aggregation and per-item detail endpoints for both hotels and event venues, and admin-only create/update/delete endpoints, built from a single shared controller factory to keep hotel and venue logic consistent.

4.3.3 Availability Module

Computes, per hotel room type, how many rooms remain free for a given date range by subtracting active (Confirmed, or still-held Pending) bookings from total room stock; for venues, checks whether the venue already has an active booking on the requested date.

4.3.4 Booking Module

Validates the requested dates/guests/room or venue capacity, re-checks availability at the moment of booking to prevent race conditions, calculates subtotal, tax (12% GST for hotels, 18% for venues) and total, and creates a booking in the "Pending" state with a unique booking code.

4.3.5 Payment Module

Simulates a payment gateway: validates the chosen method's input (UPI ID format, card number/expiry/CVV, bank selection, wallet + mobile number), introduces a short artificial delay to mimic bank communication, randomly/deterministically triggers declines for designated test values, and on success marks the booking "Confirmed" and creates a Payment record; the pay-at-property option confirms the booking immediately without any online transaction.

4.3.6 Review Module

Allows a user to post one review per hotel/venue, but only if they hold a Confirmed booking for that item; recalculates and stores a blended average rating whenever a review is added, edited or removed.

4.3.7 Admin Module

Aggregates dashboard statistics (bookings by status, revenue, payments by method, recent activity) and exposes management endpoints for hotels, venues, bookings, payments, reviews and users, including role changes and account deletion.

4.4 Database Design

The database is document-oriented (MongoDB) and consists of six primary collections, modelled with Mongoose schemas as summarised below.

4.4.1 User

Field	Type	Description
name	String	Full name of the user
userId	String (unique)	Login identifier chosen by the user
password	String (hashed)	bcrypt-hashed password
phone	String	Contact number (optional)
role	String (enum)	"user" or "admin"

4.4.2 Hotel (embeds Room sub-documents)

Field	Type	Description
name, city, state, address	String	Property identity and location
description, images[], amenities[]	String / Array	Descriptive content
propertyType, stars	String, Number	Hotel / Resort / Villa / Homestay etc., 1–5 stars
rooms[]	Array of sub-document	name, bed, price, capacity, totalRooms per room type
priceFrom	Number	Auto-computed lowest room price
rating, reviewCount, bookingCount	Number	Live aggregate values

4.4.3 Venue

Field	Type	Description
name, city, state, address	String	Venue identity and location
capacity	Number	Maximum guests
facilities[], eventTypes[]	Array	Amenities and supported event categories
pricePerDay	Number	Booking price per day
rating, reviewCount, bookingCount	Number	Live aggregate values

4.4.4 Booking

Field	Type	Description
bookingCode	String (unique)	Human-readable reference code
user, hotel/venue	ObjectId (ref)	References to related documents
type	String (enum)	"hotel" or "venue"
checkIn/checkOut or eventDate	Date	Stay dates or event date
roomType, rooms / eventType, guests	String / Number	Selection details
subtotal, tax, amount	Number	Pricing breakdown
status	String (enum)	Pending / Confirmed / Cancelled
paymentStatus, paymentMethod	String	Unpaid / Paid / Pay at property / Refunded

4.4.5 Payment

Field	Type	Description
booking, user	ObjectId (ref)	Linked booking and payer
method	String (enum)	upi / card / netbanking / wallet / cod
methodDetail	String	Masked identifier (e.g., last 4 card digits)
amount	Number	Amount charged
status	String (enum)	Success / Failed / Pending / Refunded
transactionId	String	Simulated gateway transaction reference

4.4.6 Review

Field	Type	Description
user, hotel/venue	ObjectId (ref)	Reviewer and reviewed item
rating	Number (1–5)	Star rating
comment	String	Review text

4.5 API Design Summary

Module	Representative Endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me, PUT /api/auth/profile
Catalogue	GET /api/hotels, GET /api/hotels/:id, GET /api/hotels/:id/availability (same for /venues)
Booking	POST /api/bookings, GET /api/bookings/mine, PUT /api/bookings/:id/cancel
Payment	POST /api/payments/pay, GET /api/payments/mine
Review	GET /api/reviews, POST /api/reviews, PUT /:id, DELETE /:id
Admin	GET /api/admin/stats, GET /api/admin/users, PUT /api/admin/users/:id/role

Chapter 5: Technology Stack

Layer	Technology	Purpose
Frontend	React.js 18 (Vite)	Component-based single-page application
Frontend	React Router DOM	Client-side routing and route protection
Frontend	Tailwind CSS	Utility-first responsive styling
Frontend	Axios	HTTP client for calling the REST API
Backend	Node.js	JavaScript runtime for the server
Backend	Express.js	REST API framework and routing/middleware
Backend	Mongoose	Object Data Modelling (ODM) for MongoDB
Backend	jsonwebtoken (JWT)	Stateless authentication tokens
Backend	bcryptjs	One-way password hashing
Backend	cors, morgan, dotenv	Cross-origin support, request logging, environment configuration
Database	MongoDB (Atlas / local)	Document-oriented NoSQL data store
Tooling	Git & GitHub	Version control and source hosting
Tooling	VS Code	Development environment
Deployment	Render / Railway / Vercel / Netlify, MongoDB Atlas	Cloud hosting for API, frontend and database

Chapter 6: Implementation

6.1 Backend Implementation

The backend follows an MVC-inspired structure with Mongoose models, Express route handlers ("controllers"), and route definitions kept separate. Cross-cutting concerns such as authentication and error handling are implemented as reusable middleware. A representative excerpt of the User model, which hashes passwords before saving, is shown below.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  userId: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 4 },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
}, { timestamps: true });

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

Fig. 6.1 – User schema with pre-save password hashing

Availability is computed on demand rather than stored, by counting rooms consumed by active bookings that overlap the requested date range:

```
exports.hotelAvailability = async (hotel, checkIn, checkOut, excludeId) => {
  const bookings = await Booking.find({
    type: 'hotel', hotel: hotel._id,
    checkIn: { $lt: checkOut }, checkOut: { $gt: checkIn },
    ...activeBookingFilter(), // Confirmed, or Pending within the 15-minute hold
  });
  const result = {};
  hotel.rooms.forEach((r) => {
    const used = bookings.filter((b) => b.roomType === r.name)
      .reduce((s, b) => s + b.rooms, 0);
    result[r.name] = Math.max(0, r.totalRooms - used);
  });
  return result;
};
```

Fig. 6.2 – Date-range aware hotel room availability calculation

Authentication middleware verifies the JWT on protected routes and attaches the authenticated user to the request object; a second middleware restricts admin-only routes:

```
exports.protect = asyncHandler(async (req, res, next) => {
  const header = req.headers.authorization || '';
  if (!header.startsWith('Bearer ')) throw httpError(401, 'Please sign in to continue');
  const decoded = jwt.verify(header.split(' ')[1], process.env.JWT_SECRET);
  const user = await User.findById(decoded.id);
  if (!user) throw httpError(401, 'Account not found');
  req.user = user;
  next();
});
```

```
exports.adminOnly = (req, res, next) =>
  req.user?.role === 'admin' ? next() : next(httpError(403, 'Admin access only'));
```

Fig. 6.3 – JWT verification and role-based access control middleware

6.2 Payment Simulation Logic

Since the project does not use a licensed payment aggregator, a demo payment module validates the input format for each method and simulates gateway approval or decline (for example, a card number of 4000 0000 0000 0002 or a UPI ID ending in "@fail" is treated as a decline), so the full success and failure paths can be demonstrated safely:

```
if (method === 'upi') {
  if (!/^[\w.-]{2,}@[a-zA-Z]{2,}$/.test(vpa))
    throw httpError(400, 'Enter a valid UPI ID, for example name@okhdfc');
  if (vpa.endsWith('@fail'))
    return { detail: mask(vpa), declined: 'UPI request was declined by the bank' };
}
// ... on success:
booking.status = 'Confirmed';
booking.paymentStatus = 'Paid';
await Payment.create({ booking: booking._id, method, amount, status: 'Success',
transactionId });
```

Fig. 6.4 – Simulated UPI validation and booking confirmation on success

6.3 Frontend Implementation

The React frontend uses React Router to separate fully public routes (home, search, listing details) from protected routes that require a signed-in user (booking, payment, confirmation, my bookings, profile, reviews) and admin-only routes (the dashboard). A lightweight guard component redirects unauthenticated users to the login page only when they reach a protected route, preserving the page they were trying to reach so they return to it after signing in:

```
export function Protected({ children }) {
  const { user, loading } = useAuth();
  const loc = useLocation();
  if (loading) return <Spinner />;
  if (!user) return <Navigate to="/login" replace
    state={{ from: loc.pathname + loc.search }} />;
  return children;
}
```

Fig. 6.5 – Route guard used to defer sign-in until checkout

Global authentication state is exposed through a React Context (AuthContext) that stores the JWT in localStorage and re-hydrates the current user on page load, while a ToastContext provides consistent success/error notifications across the application.

6.4 Security Measures Implemented

- Passwords are never stored in plain text; bcrypt with a salt round of 10 is used for hashing.
- JWTs are signed with a server-side secret and verified on every protected request; tokens are stateless and carry only the user ID.

- All state-changing operations (create/update/delete/cancel/pay) are re-validated on the server, not trusted from client input alone.
- Role checks (adminOnly middleware) guard every administrative endpoint independently of the frontend UI.
- A centralised error-handling middleware normalises all error responses and avoids leaking internal stack traces to the client.

Chapter 7: Testing

7.1 Testing Approach

The system was tested using a combination of manual functional testing (exercising each user journey end-to-end through the UI) and direct API testing (verifying request validation, status codes and response payloads for each endpoint). Both positive test cases (expected valid input) and negative test cases (invalid input, unauthorized access, expired holds, sold-out inventory) were exercised for every major module: authentication, catalogue browsing, booking, payment and reviews.

7.2 Test Cases and Results

Test ID	Test Description	Expected Result	Result
TC-01	Browse hotels without logging in	Home and listing pages load and display data with no account	Pass
TC-02	Register with a user ID shorter than 3 characters	Registration is rejected with a clear message	Pass
TC-03	Register with an already-used user ID	Registration is rejected as "user ID is taken"	Pass
TC-04	Log in with correct user ID and password	JWT is issued and the user is redirected to the intended page	Pass
TC-05	Log in with incorrect password	Login is rejected as "Wrong user ID or password"	Pass
TC-06	Book the last remaining room for overlapping dates from two sessions	Second booking attempt is rejected as sold out	Pass
TC-07	Leave a Pending booking unpaid for over 15 minutes	Room hold is released; payment attempt is rejected as expired	Pass
TC-08	Pay using a valid UPI ID	Payment succeeds; booking status becomes Confirmed	Pass
TC-09	Pay using UPI ID ending in "@fail"	Payment is declined with an on-screen reason	Pass
TC-10	Pay using card number 4000 0000 0000 0002	Payment is declined as a simulated card decline	Pass
TC-11	Choose "Pay at property"	Booking is confirmed immediately with no online charge	Pass
TC-12	Cancel a booking more than 24 hours before check-in	Booking is cancelled and, if paid, marked Refunded	Pass
TC-13	Cancel a booking within 24 hours of check-in	Cancellation is rejected with an explanatory message	Pass
TC-14	Submit a review without a confirmed booking for that hotel	Review submission is rejected	Pass

Test ID	Test Description	Expected Result	Result
TC-15	Submit a duplicate review for the same hotel	Second review is rejected; user is asked to edit the existing one	Pass
TC-16	Non-admin user accesses an admin API endpoint	Request is rejected with a 403 Admin access only error	Pass
TC-17	Admin adds a new hotel with room types	Hotel appears in search results with correct starting price	Pass

7.3 Summary of Testing

All listed test cases produced the expected result at the time of testing. Server-side validation (in addition to client-side form validation) proved effective at rejecting invalid states such as double bookings, expired payment windows and unauthorized review submissions, confirming that the business rules are enforced at the API layer rather than relying solely on the user interface.

Chapter 8: Results and Discussion

The completed StayEvent application successfully demonstrates all the objectives set out in Chapter

1. The following screens were implemented and verified during development and testing:

- Home page with a destination search bar, top destinations, popular landmarks, top-picks, best-reviewed and most-booked carousels, and a full footer — all visible without signing in.
- Hotel and venue search/listing pages with sidebar filters (price, rating, star rating, amenities/facilities, property type, event type) and sortable results.
- Hotel and venue detail pages showing an image gallery, description, amenities/facilities, live availability by date, room-type pricing table (for hotels) and the guest reviews section.
- A simple registration/login screen using only a user ID and password, reached only when the visitor is ready to book.
- A three-step booking flow: guest details → payment method selection → confirmation, with a visible countdown showing the remaining time on the 15-minute inventory hold.
- A payment screen supporting UPI, card, net banking, wallet and pay-at-property, with an animated processing state and an animated success confirmation.
- A confirmation page showing the booking code, transaction details and a print/save option.
- A "My Bookings" page with status filters (All/Confirmed/Pending/Cancelled) and cancel/pay-now actions.
- An administrator dashboard with summary statistics, and dedicated management screens for hotels, venues, bookings, payments, reviews and users.

[Insert screenshots of each of the above screens here before final submission, captured from the running application at <http://localhost:5173>, with a short caption under each figure, e.g. "Fig. 8.1 – Home page".]

8.1 Discussion

The open-browsing design (allowing exploration without an account) matched the intended goal of reducing friction, since sign-in is requested only immediately before a booking is created, at which point the visitor's intent is already established. The 15-minute inventory hold combined with server-side availability recalculation at both booking creation and payment time was found to reliably prevent double bookings even when tested with overlapping requests. The simulated payment gateway, while not connected to a real bank, closely reproduces the UX of a production checkout (method selection, input validation, a processing delay, and success/failure branches), which was sufficient to demonstrate a complete and realistic booking-to-payment journey for academic evaluation purposes.

Chapter 9: Conclusion and Future Scope

9.1 Conclusion

This project successfully designed and implemented StayEvent, a full-stack MERN web application that unifies hotel room and event venue booking into a single platform. The system allows unrestricted browsing, requires only a lightweight user ID and password to sign in, enforces correct date-based availability to prevent double bookings, and offers a realistic, multi-method simulated payment experience. A complete administrative dashboard was also implemented to manage the catalogue, bookings, payments, reviews and users. The project demonstrates practical application of REST API design, JWT-based authentication, MongoDB document modelling, and modern React development practices, and meets the objectives defined at the outset of the project.

9.2 Future Scope

- Integrate a licensed payment aggregator (e.g., Razorpay or Stripe, in test or live mode) in place of the simulated gateway.
- Add SMS/email/WhatsApp notifications for booking confirmation, payment receipts and cancellation reminders.
- Add map-based search and location picking using a mapping API.
- Introduce a loyalty/rewards program and coupon/discount codes.
- Add multi-language and multi-currency support for a wider audience.
- Allow property owners to self-register and manage their own listings (a marketplace model) rather than only platform administrators adding listings.
- Add analytics dashboards with charts for revenue trends, occupancy rates and customer demographics.

References

1. React documentation – <https://react.dev>
2. Express.js documentation – <https://expressjs.com>
3. MongoDB documentation – <https://www.mongodb.com/docs>
4. Mongoose ODM documentation – <https://mongoosejs.com>
5. Node.js documentation – <https://nodejs.org/en/docs>
6. JSON Web Tokens (JWT) introduction – <https://jwt.io/introduction>
7. bcrypt / bcryptjs package documentation – <https://www.npmjs.com/package/bcryptjs>
8. Tailwind CSS documentation – <https://tailwindcss.com/docs>
9. Vite documentation – <https://vitejs.dev>
10. Booking.com – referenced for UI/UX patterns of hotel search and listing pages – <https://www.booking.com>
11. MDN Web Docs – <https://developer.mozilla.org>

Appendix A: Sample Backend Route Configuration

```
router.post('/bookings', protect, booking.create);
router.get('/bookings/mine', protect, booking.mine);
router.put('/bookings/:id/cancel', protect, booking.cancel);
router.post('/payments/pay', protect, payment.pay);
router.get('/admin/stats', protect, adminOnly, admin.stats);
```

Appendix Fig. A.1 – Excerpt of API route definitions

Appendix B: Demo Access & Deployment Summary

B.1 Demo Credentials (for evaluation only – change before any public deployment)

Role	User ID	Password
Customer	demo	demo123
Administrator	admin	admin123

B.2 Local Setup

1. Install MongoDB locally, or create a free MongoDB Atlas cluster.
2. In the "server" folder, run "npm install" then "npm run dev" (starts the API on port 5000).
3. In the "client" folder, run "npm install" then "npm run dev" (starts the website on port 5173).
4. Open <http://localhost:5173> in a browser; sample hotels, venues and demo accounts load automatically on first run.

B.3 Suggested Free Deployment

Frontend and backend can each be deployed as a free Render (or Railway) service, with the MongoDB database hosted on a free MongoDB Atlas cluster, requiring no code changes to the project as built.